IEM UTM STUDENT SECTION

UTM UNIVERSITI TEKNOLOGI MALAYSIA

IEM Connect

Sprint 2 Presentation

Presented by: Group GGEZ

Date: 1 December 2025

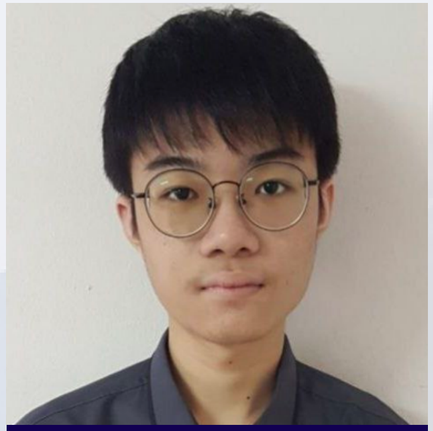
Play

Our Team

Chuah Chun Yi

Scrum Master

Lead Coordination & Stakeholder Communication



Chong Jun Hong

Developer 1

Backend Development

Muhammad Ariff bin Baharuddin

Developer 2

Frontend Development

Huzsa Syahyzal Haiqal bin Hussain

Developer 3

Frontend Development

Hrishi Vengalsetti

Developer 4

Backend Development

Sprint Overview

Sprint Goal

- To deliver the major functional components of the IEM Connect system by enabling users and admin to manage profiles and events seamlessly.

Sprint Duration

- 2 Weeks (Design + Development + Integra+ Testing)

MVP Objective

- Build the core features related to profile customization and event handling for both admins and users.
- Deliver event announcement features for both admins and users

Sprint Backlog Summary

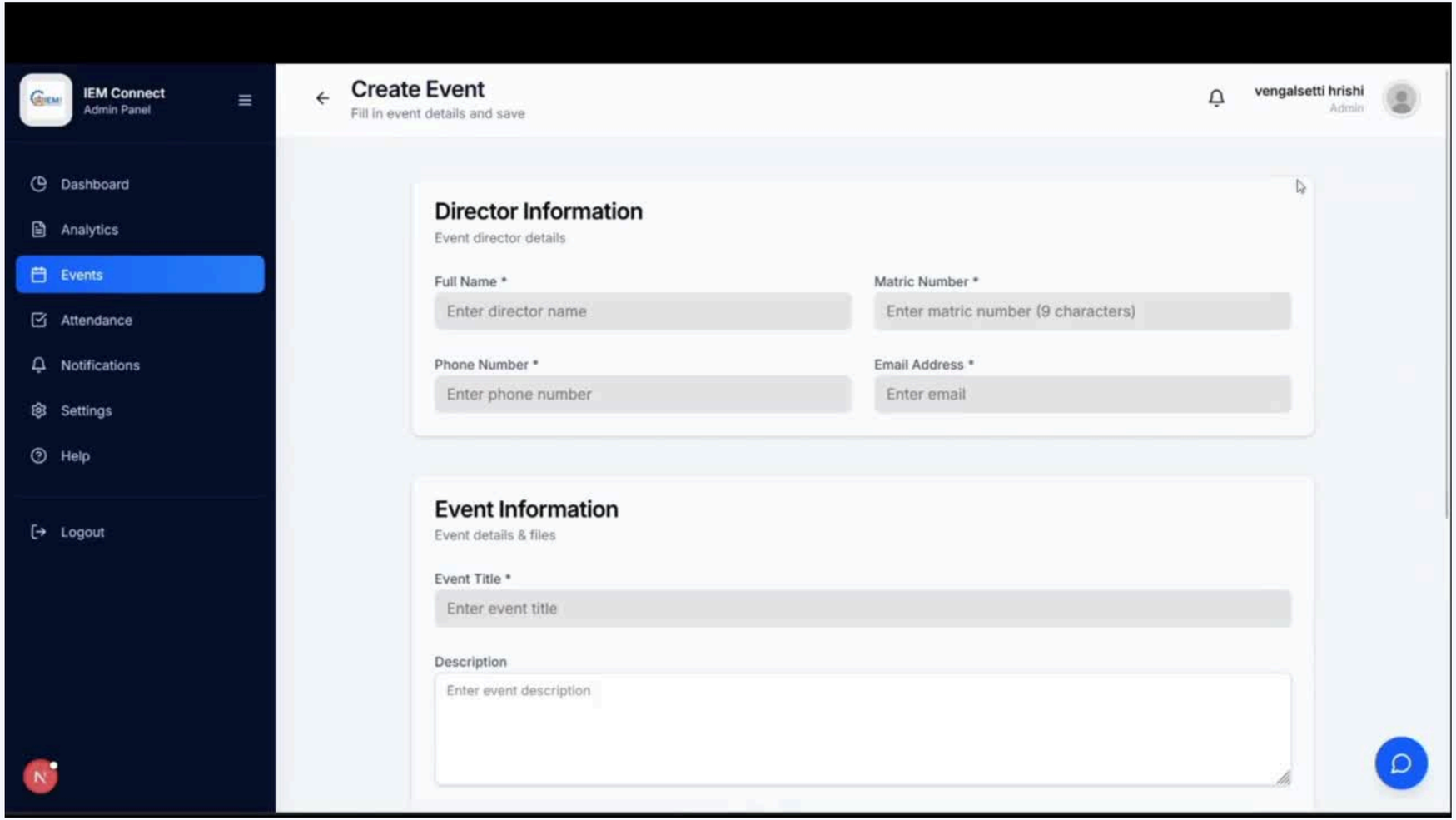
Main User Stories Implemented:

1. As a member, I want to edit and update my profile details and information so that my personal information and details remain accurate.
2. As an admin, I want to create, edit, and manage events so that members can easily view and register for them.
3. As an admin, I want to post event announcements so that all members can stay informed in one centralized platform.

Deliverables:

- Profile Management Module
- Event Creation & Management Module
- Event Announcement Page

MVP Demonstration



Client Feedback

Key Comments Received:

1. Sprint 2 MVP meets functional expectations.
2. Improve UI attractiveness and consistency.
3. Add profile picture upload
4. Implement automatic reminders for upcoming events
5. Add dismiss (×) button for notifications
6. Display events using card layout for better organization.

Action Items for Next Sprint:

- Enhance overall UI design and responsiveness.
- Add profile picture upload feature.
- Add auto-notifications and notification dismissal.

Code Hightlights

Core Components

backend/controllers/authController.js:

- Handle updates on user profile
- Handle change password
- Handle delete account

backend/controllers/eventController.js:

- Handle creation and retrieval of events
- Handle editing of events
- Get specific event information

backend/controllers/notificationController.js:

- Handle creation and retrieval of notifications
- Handle sending of event annoncements

Example Logic

1 Profile Update Logic

```
export const updateProfile = async (req, res) => {
  try {
    const schema = Joi.object({
      name: Joi.string().min(1).optional(),
      bio: Joi.string().max(500).allow("", null).optional(),
    });
    const { error, value } = schema.validate(req.body);
    if (error) {
      return res.status(400).json({ error: error.details[0].message });
    }
    const user = await User.findByPk(req.user.id);
    if (!user) {
      return res.status(404).json({ error: "User not found" });
    }
    const updateData = {};
    if (value.name !== undefined) updateData.name = value.name;
    if (value.bio !== undefined) updateData.bio = value.bio;

    await user.update(updateData);
    res.json({ msg: "Profile updated successfully", user });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

- Validates input
- Updates the user's profile fields
- Returns a success response

Example Logic

2 Event Creation Logic

```
export const createEvent = async (req, res) => {
  try {
    const schema = Joi.object({
      title: Joi.string().required(),
      description: Joi.string().required(),
      date: Joi.date().required(),
      location: Joi.string().required(),
      // ... other event fields
    });
    const { error, value } = schema.validate(req.body);
    if (error) {
      return res.status(400).json({ error: error.details[0].message });
    }
    const event = await Event.create({
      title: value.title,
      description: value.description,
      date: value.date,
      location: value.location,
      createdBy: req.user.id,
    });
    res.status(201).json({
      message: "Event created successfully",
      event: event,
    });
  } catch (error) {
    console.error("Create event error:", error);
    res.status(500).json({ error: "Failed to create event" });
  }
};
```

- Validates event data
- Creates a new event record
- Returns a success response

Example Logic

3 Event Announcement Logic

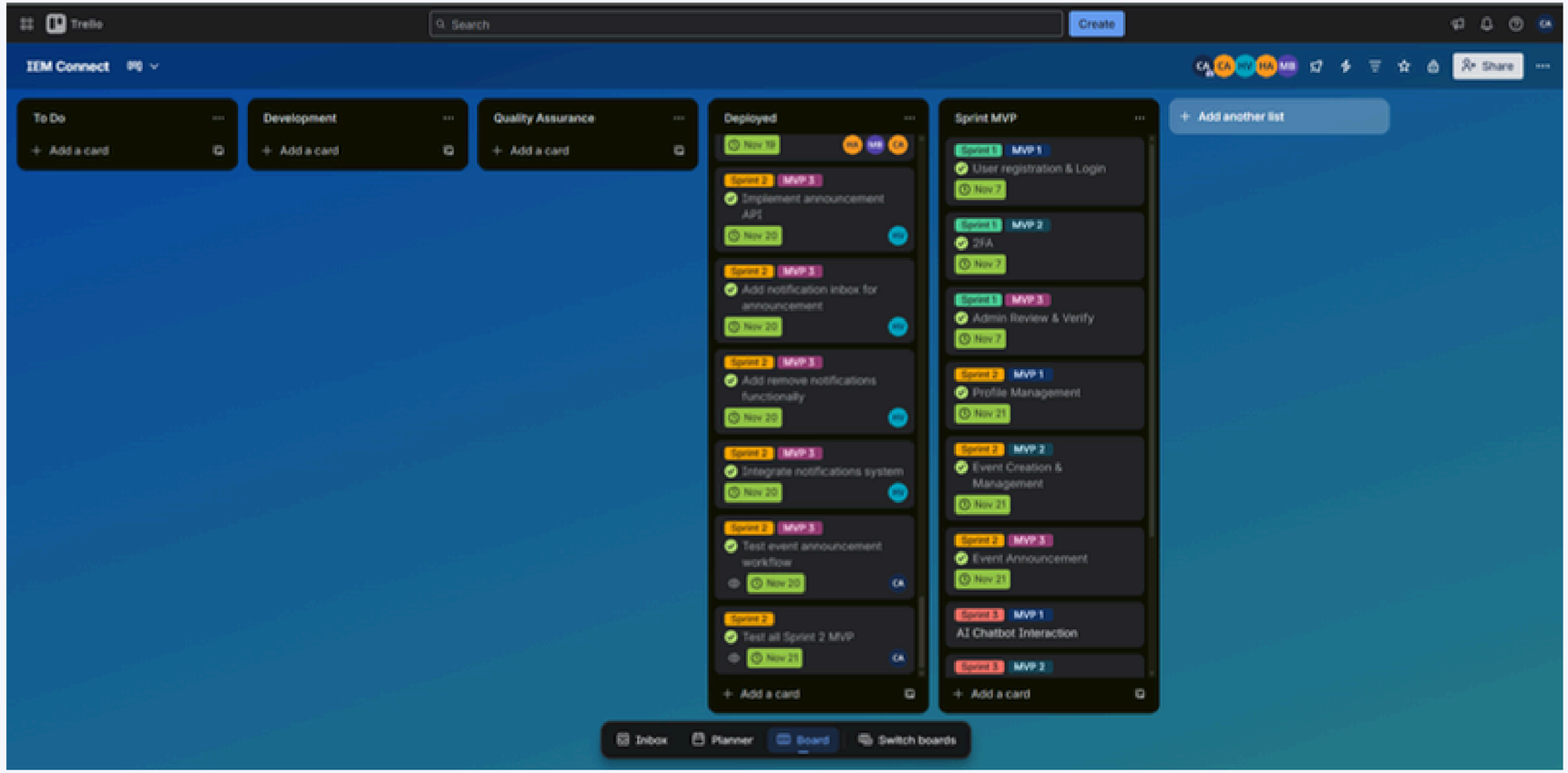
```
export const sendEventAnnouncement = async (req, res) => {
 try {
  const { id } = req.params;
  const { subject, message, sendEmail = true } = req.body;
  if (!subject || !message) {
   return res
    .status(400)
    .json({ error: "Subject and message are required" });
  }
  const event = await Event.findByPk(id);
  if (!event) {
   return res.status(404).json({ error: "Event not found" });
  }
  const registrations = await EventRegistration.findAll({
   where: {
    event_id: id,
    status: "registered", // Only notify registered participants
   },
   attributes: ["user_id"],
  });
  if (registrations.length === 0) {
   return res.json({
    message: "No registered participants found for this event",
    sent: 0,
   });
  }
```

```
  const userIds = registrations.map((reg) => reg.user_id);
  const result = await NotificationService.notifyUsers(
   userIds,
   subject,
   message,
   "announcements", // notification type
   sendEmail // force email if explicitly requested by admin
  );
  res.json({
   message: `Announcement sent to ${result.success} participant(s)`,
   event: {
    id: event.id,
    title: event.title,
   },
   sent: result.success,
   failed: result.failed,
   total: result.total,
  });
 } catch (error) {
  console.error("Send event announcement error:", error);
  res.status(500).json({ error: "Failed to send announcement" });
 }
};
```

- Retrieves an event’s registered participants,
- Sends them the announcement
- Returns the delivery status

Project Performance Metrics

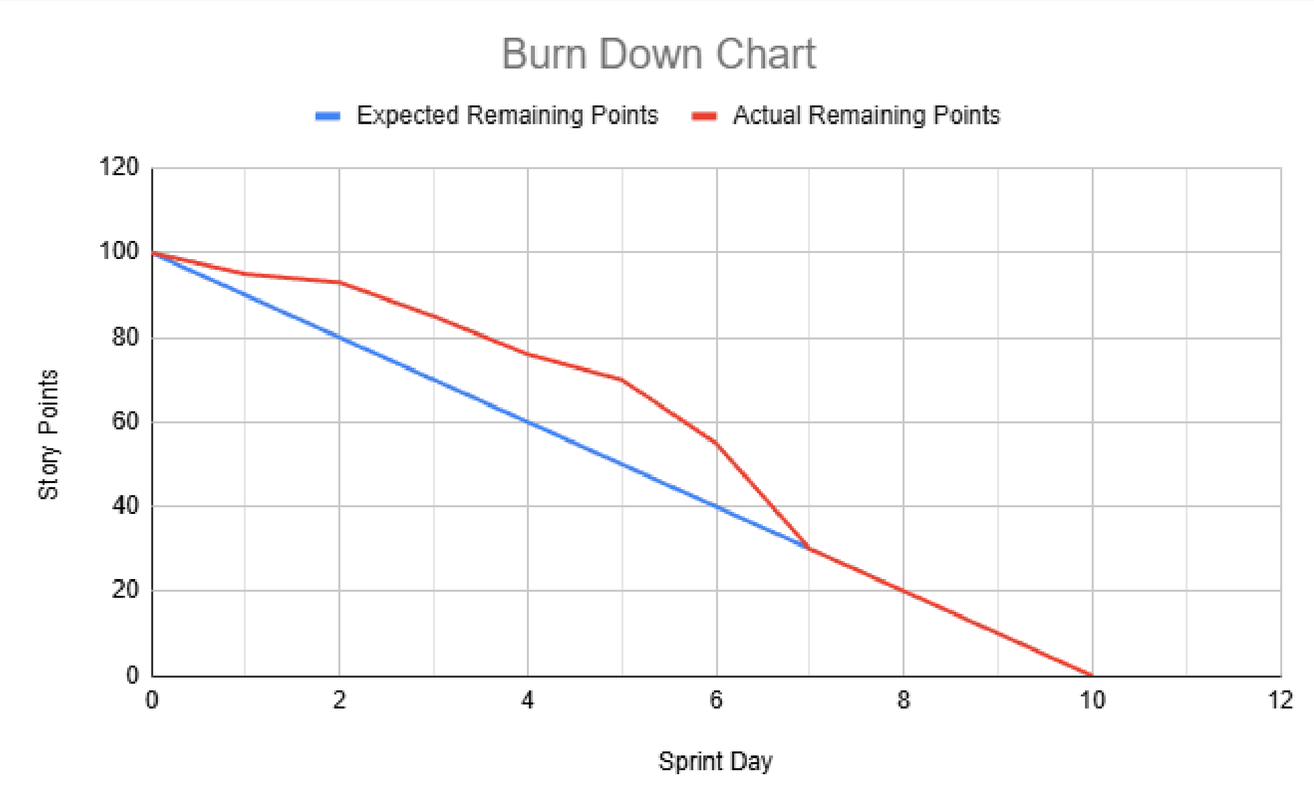
Kanban Board



Sprint 2
Total tasks: 26
Completed: 26
In progress: 0

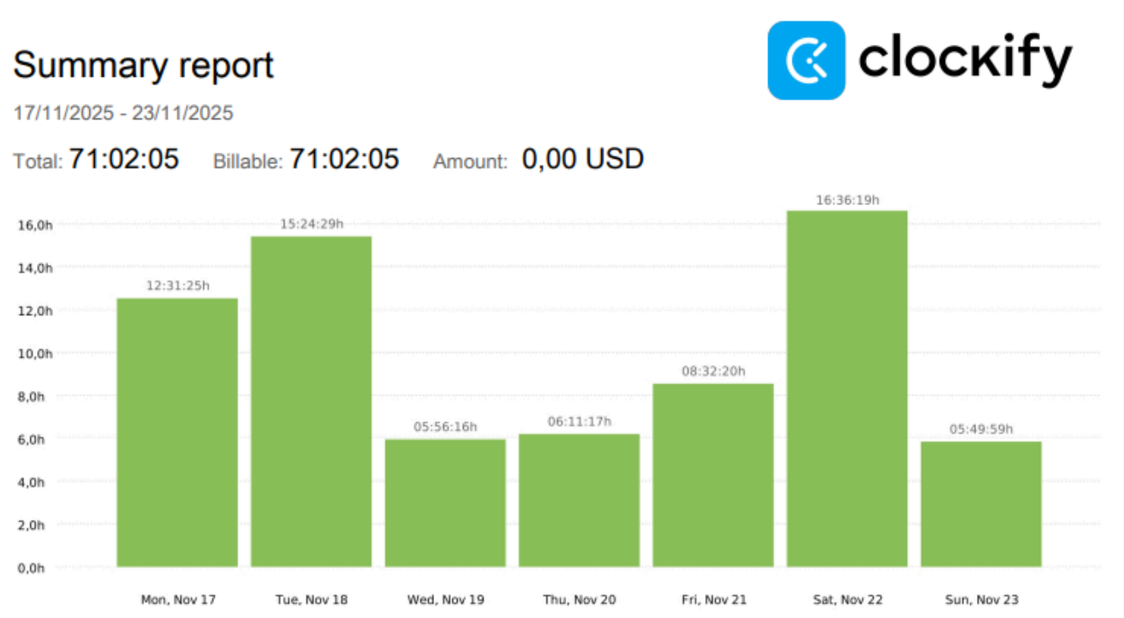
Project
Total tasks: 11
Completed: 6
In progress: 5

Burndown Chart



Our team initially lagged but caught up and completed all story points by the end of the sprint.

Clockify Summary



Total hours logged: 138.17 hours
Average per member: 27.63 hours

Next Sprint Focus

Sprint 3 Objectives:

- Integrate AI chatbot for FAQ assistance
- Develop dashboard analytics for admin
- Implement event report generation

Target Outcome:

- Transform IEM Connect from a management tool into an intelligent system providing analytics insights and automated support.

Conclusion

Summary:

- Sprint 2 successfully delivered a working MVP with complete profile management, fully functional event creation & management with attendance tracking and event announcement functionalities.

Next Steps:

- Implement client feedback
- Improve system UI and dashboards
- Prepare testing scripts for next sprint
- Begin research on AI chatbot APIs

Thank You

Contact Us

Chuah Chun Yi

Scrum Master of Group GGEZ

+60 1157678402

iemconnect1@gmail.com